

MÓDULO 00

Diseño de Producto MVP

Design Sprint + Material Design 3 para apps Flutter. Define y valida qué construir antes de escribir una línea de código, y diseña la interfaz con el mismo sistema que usa Flutter de forma nativa.

METODOLOGÍA

**GV Design Sprint +
Material Design 3**

BACKEND

**Listo para implementar
con Supabase (BaaS)**

NIVEL

**Producto / Diseño /
Flutter**

TIEMPO ESTIMADO

**5 días de Sprint +
implementación M3**

Design Sprint

Material Design 3

Flutter

MVP

Contenido

10 capítulos · Design Sprint + Material Design 3 para MVP Flutter

PARTE 1 · DESIGN SPRINT

01 01 — Introducción al Design Sprint para MVP Mobile

02 02 — Understand + Define

03 03 — Sketch + Decide

PARTE 2 · MATERIAL DESIGN 3

04 04 — Fundamentos de Material Design 3

05 05 — Componentes M3 Clave para Mobile

PARTE 3 · PROTOTIPADO Y VALIDACIÓN

06 06 — Prototipado + Validación

PARTE 4 · IMPLEMENTACIÓN FLUTTER

07 07 — M3 a Flutter: Implementación

08 08 — Template de proyecto

PARTE 5 · CASO INTEGRADOR

09 09 — Caso completo: App de Reservas

PARTE 6 · CIERRE

BI Bibliografía y recursos oficiales

01 — Introducción al Design Sprint para MVP Mobile

¿Qué es un Design Sprint?

El **Design Sprint** es una metodología creada por Google para responder preguntas críticas de negocio a través del diseño, prototipado y testing con usuarios reales en **5 días**. Fue desarrollada por Jake Knapp en Google Ventures (GV) y hoy está documentada de forma oficial en gv.com/sprint. El antiguo Design Sprint Kit de Google (con el que trabajó este tema durante años) sigue accesible como archivo histórico, pero ya no recibe soporte activo.

No es una metodología más. Es el estándar que Google usa internamente para validar productos antes de invertir en desarrollo.

Las 6 fases

LUNES	MARTES	MIÉRCOLES	JUEVES	VIERNES
Understand →	Define →	Sketch →	Decide →	Prototype → Validate
(Entender)	(Definir)	(Bosquejar)	(Decidir)	(Prototipar) (Validar)

Fase	Duración	Objetivo
Understand	1 día	Mapear el problema, alinear al equipo, recopilar información
Define	(incluido en Understand)	Definir el foco del sprint, metas y métricas de éxito
Sketch	1 día	Generar soluciones individuales en papel
Decide	1 día	Criticar, votar y decidir la mejor solución
Prototype	1 día	Construir un prototipo realista (pero falso)
Validate	1 día	Testear con 5 usuarios reales

¿Por qué funciona?

1. **Comprime meses en 5 días**: lo que normalmente toma investigación, proof-of-concept y discusiones interminables se reduce a una semana enfocada.
2. **Pone al usuario en el centro**: el viernes tienes evidencia real, no suposiciones.
3. **Alinea al equipo**: diseño, producto e ingeniería trabajan juntos desde el día 1.

Design Sprint vs. metodologías tradicionales

Aspecto	Design Sprint	Agile/Scrum tradicional
Horizonte	Descubrimiento (descubrir qué construir)	Entrega (cómo construir ya definido)
Validación	Antes de escribir código	Después de escribir código
Riesgo	Bajo (prototipo rápido)	Alto (inversión en desarrollo)
Usuarios	5 usuarios en 1 día	Usuarios reales en producción
Cambio de dirección	Gratis (solo papel)	Costoso (código desechado)

Adaptación para MVP mobile con Flutter

Un Design Sprint clásico asume que prototipas en herramientas como Figma o Keynote. Para un MVP mobile con Flutter, la adaptación es:

1. **Understand + Define**: mismos ejercicios (mapa, goal, expert interviews). El entregable es un **documento de alcance del MVP** con las pantallas críticas identificadas.
2. **Sketch + Decide**: mismos ejercicios. El entregable es un **storyboard** del flujo principal de la app.
3. **Prototype**: aquí puedes elegir:
 - **Opción A (recomendada para no-devs)**: Figma con el [kit de M3](#). Rápido, sin código.
 - **Opción B (para devs)**: Flutter con el template M3 de este módulo. Más realista, más lento.
 - **Opción C (híbrida)**: Prototipo en Figma para validar el jueves, y mientras tanto preparar la base en Flutter con el template.
4. **Validate**: pruebas con usuarios sobre el prototipo. El entregable es un **informe de validación** con hallazgos y decisiones.

Antes del Sprint: el equipo

Para un Sprint de app móvil necesitas:

Rol	Cantidad	Responsabilidad
Facilitador	1	Cronometra, guía los ejercicios, no opina
Decisor	1	Dueño del producto, tiene la última palabra
Diseñador UI/UX	1	Domina M3, Figma, patrones mobile
Flutter Dev	1	Conoce viabilidad técnica, costos de implementación
Experto en negocio	1	Conoce el mercado, los usuarios, la competencia

Mínimo viable: Facilitador + Decisor + Diseñador. Si eres solo tú, puedes hacer un Sprint en solitario (más lento, pero funciona).

¿Cuándo NO hacer un Design Sprint?

- Cuando el problema ya está muy definido y solo falta implementar
 - Cuando no tienes acceso a usuarios para validar
 - Cuando el equipo no puede comprometer 5 días completos
 - Cuando necesitas un producto completo (el Sprint solo da un MVP validado)
-

Siguiente: 02 — Understand + Define

02 — Understand + Define

Día 1: Entender el problema y definir el foco

Estas dos fases ocurren en el **mismo día**. La mañana es para entender, la tarde para definir.

Fase 1: Understand

El objetivo es que todo el equipo tenga el mismo nivel de información sobre el problema, el usuario y el negocio.

1.1. Ejercicio: Start at the End

Define el **goal a largo plazo** del producto.

Pregunta: "¿Qué estará pasando cuando nuestro producto sea un éxito?"

Ejemplo: "Los usuarios podrán reservar una cancha de tenis en menos de 30 segundos desde su teléfono, sin llamar por teléfono."

El goal debe tener una **métrica** asociada:

Ejemplo: "Reducir el tiempo de reserva promedio de 5 minutos a <30 segundos"

1.2. Ejercicio: Mapa del problema

Dibuja el flujo del usuario **actual** (sin tu producto). Identifica puntos de fricción.

[Usuario quiere jugar tenis]

- Abre Google Maps → Busca canchas → Encuentra teléfono
- Llama → Ocupado → Vuelve a llamar → Pregunta disponibilidad
- "Sí, hay a las 5pm" → "OK, reserve" → Llega y la cancha está ocupada
- ← FRUSTRACIÓN ←

Los puntos de fricción son **oportunidades de diseño**.

1.3. Ejercicio: Expert Interviews

Cada miembro del equipo comparte lo que sabe. Formato:

Persona	Aporta
Decisor	Visión de negocio, restricciones, stakeholders
Diseñador	Patrones de UX, competencia, tendencias mobile
Dev	Viabilidad técnica, costos, tiempos
Experto	Datos de mercado, investigación de usuarios

Preguntas clave:

- ¿Qué sabemos con certeza?
- ¿Qué estamos asumiendo?
- ¿Qué necesitaríamos saber para tener confianza?

1.4. Ejercicio: How Might We (HMW)

Reformula cada punto de fricción como una oportunidad:

Fricción: "Llamar por teléfono es lento y poco confiable" HMW: "¿Cómo podríamos permitir reservar sin hablar con nadie?"

Fricción: "No saber si la cancha estará disponible al llegar" HMW: "¿Cómo podríamos garantizar disponibilidad en tiempo real?"

Fase 2: Define

Con toda la información sobre la mesa, es hora de **acotar**.

2.1. Ejercicio: The Golden Path

Define el **camino ideal** del usuario dentro del MVP. No más de 3-4 pantallas.

Pantalla 1: Home → "Buscar canchas"
 Pantalla 2: Lista de canchas → "Seleccionar fecha y hora"
 Pantalla 3: Confirmar reserva → "Pagar"
 Pantalla 4: Confirmación → "Listo, llegas a las 5pm"

El Golden Path es el **flujo crítico** del MVP. Todo lo demás: registro, perfil, historial, etc. son **post-MVP**.

2.2. Ejercicio: Definir el alcance del MVP

El MVP no es "lo mínimo para salir", sino **lo mínimo para validar valor**.

Usa la matriz **Impacto vs. Esfuerzo**:

Impacto ↑

[Hacer primero]	[Hacer después]	
Golden Path	Historial	
Reserva rápida	Favoritos	

[Último]	[Evitar]	
Chat en vivo	Modo oscuro	
Landing page	Animaciones	
→		Esfuerzo

Regla de oro del MVP mobile:

Si no está en el Golden Path, no entra en el MVP.

2.3. Sprint Goal

Redacta el objetivo del Sprint en una frase:

"Validar que los usuarios pueden reservar una cancha de tenis en menos de 30 segundos desde su teléfono."

Esto guiará todas las decisiones del resto de la semana.

2.4. Checklist del Día 1

- ☐ Goal a largo plazo definido con métrica
- ☐ Mapa del problema dibujado
- ☐ HMWs generados a partir de fricciones
- ☐ Golden Path definido (3-4 pantallas)
- ☐ MVP scope acotado (qué entra / qué no entra)
- ☐ Sprint Goal redactado
- ☐ Usuarios reclutados para el viernes (Validate)

Plantilla: Documento de alcance del MVP

```
---
MVP: [Nombre del producto]
Goal: [Frase única]
Métrica principal: [Ej: tiempo de reserva <30s]
---

## Golden Path
1. [Pantalla 1: acción]
2. [Pantalla 2: acción]
3. [Pantalla 3: acción]

## Incluye (MVP)
- [Feature 1]
- [Feature 2]
- [Feature 3]

## No incluye (post-MVP)
- [Feature X]
- [Feature Y]

## Supuestos a validar
1. [Supuesto 1: ej, "los usuarios quieren reservar sin hablar"]
2. [Supuesto 2]

## Criterios de éxito
- [ ] [Criterio 1]
- [ ] [Criterio 2]
```

Siguiente: 03 — Sketch + Decide

03 — Sketch + Decide

Día 2: Bosquejar soluciones → Día 3: Decidir la mejor

Fase 3: Sketch

El objetivo es que **cada miembro del equipo** genere soluciones de forma individual. No se discute, no se critica, se dibuja.

3.1. Ejercicio: Lightning Demos (mañana)

Antes de dibujar, inspírate. Cada miembro muestra 2-3 apps o productos existentes que resuelven problemas similares.

Para cada demo, responde:

- ¿Qué hace bien?
- ¿Qué harías diferente?
- ¿Qué podríamos robar (remix)?

Ejemplo para una app de reservas:

Airbnb: la navegación por fechas es excelente. **OpenTable:** la confirmación instantánea da confianza.
Uber: no necesitas registro para usarlo.

3.2. Ejercicio: Crazy 8s

Cada persona dobla una hoja en 8 partes. En **8 minutos** (1 minuto por recuadro), dibuja 8 variaciones de una pantalla clave.

Var 1	Var 2	Var 3	Var 4
Var 5	Var 6	Var 7	Var 8

No importa la calidad del dibujo. Importa la cantidad de ideas.

3.3. Ejercicio: Solution Sketch (tarde)

Cada persona crea un **boceto detallado** de su mejor solución. Formato de 3 paneles:

Panel 1: Estado actual / problema (una escena que muestre la frustración del usuario)
Panel 2: La solución (la pantalla clave del MVP)
Panel 3: El resultado (el usuario feliz, métrica lograda)

El Solution Sketch debe ser **auto-explicativo**. Mañana alguien más podría tener que entenderlo sin que le expliques nada.

Herramientas: papel y lápiz (recomendado), o tool digital como Miro/Figma.

Fase 4: Decide

El objetivo es **elegir una sola solución** para prototipar.

4.1. Ejercicio: Art Museum

Pega todos los Solution Sketches en la pared. El equipo camina en silencio y los revisa.

Cada persona tiene:

- **2 dot votes grandes** (👍): "Esto es genial, deberíamos construirlo"
- **1 dot vote pequeño** (❓): "Esto es interesante, pero tengo dudas"

4.2. Ejercicio: Heat Map + Speed Critique

El facilitador selecciona los sketches con más votos. El equipo los critica rápidamente (3 min cada uno):

Pregunta	Tiempo
¿Qué nos gusta?	1 min
¿Qué dudas tenemos?	1 min
¿Qué mejorarías?	1 min

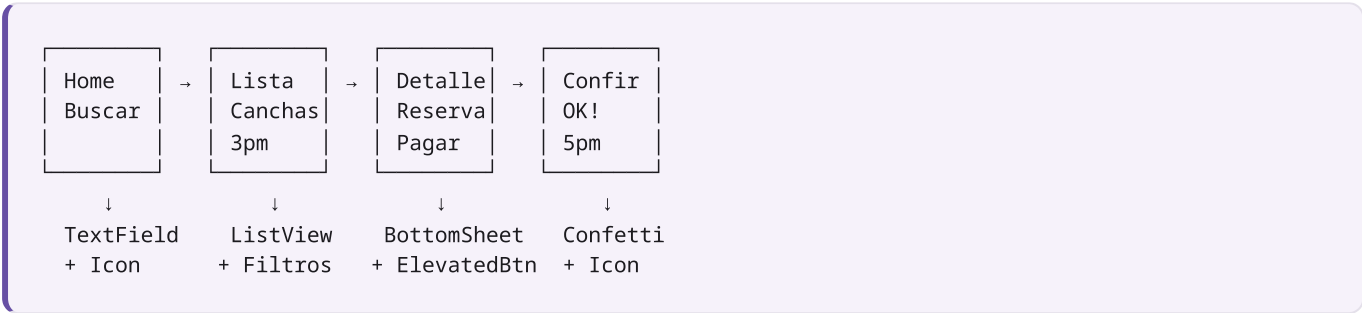
4.3. Ejercicio: Straw Poll + Decisión Final

Votación final. Cada persona tiene **1 voto** (no puede votar por el suyo).

- Si hay empate, el **Decisor** desempata.
- El Decisor puede elegir una solución híbrida ("tomamos la navegación de A y el detalle de B").

4.4. Ejercicio: Storyboard

Con la solución elegida, el diseñador (con ayuda del equipo) crea un **storyboard** detallado del prototipo. Esto es el **plano** para el Día 4.



Cada pantalla del storyboard debe incluir:

- **Layout** básico (qué va donde)
- **Componentes M3** a usar (ver módulo 05)
- **Flujo de datos** (qué pasa cuando el usuario toca)
- **Estados:** carga, vacío, error, éxito

4.5. Checklist del Día 3

- ☐ Solución única seleccionada y aprobada por el Decisor
- ☐ Storyboard detallado con todas las pantallas del Golden Path
- ☐ Componentes M3 identificados para cada pantalla
- ☐ Prototipador asignado y herramientas listas (Figma / Flutter)
- ☐ Guión de entrevista preparado para el viernes
- ☐ Usuarios confirmados para Validate (mínimo 5)

De Sketch/Decide a M3

El storyboard es el **punto** entre el Design Sprint y Material Design 3.

En el storyboard defines **qué** va en cada pantalla. En los próximos capítulos aprenderás **cómo** diseñarlo con M3.

Storyboard dice	M3 provee
"Barra de navegación abajo"	NavigationBar
"Tarjeta con producto"	Card + ListTile
"Botón principal"	FilledButton / FilledTonalButton
"Selector de fecha"	DatePickerDialog
"Campo de búsqueda"	SearchBar

04 — Fundamentos de Material Design 3

El sistema de diseño oficial de Google para apps móviles

Material Design 3 (M3) es el sistema de diseño de Google, también conocido como **Material You**. Es la evolución de Material Design y **el que Flutter usa por defecto desde la versión 3.16** (antes de esa versión era opcional activarlo).

M3 se basa en 3 subsistemas principales:

M3 Theme

- ColorScheme (color)
- TextTheme (tipografía)
- ShapeScheme (formas)

1. Color: esquemas tonales

M3 introduce el concepto de **paletas tonales**. En lugar de elegir colores individuales, eliges un **color semilla (seed)** y M3 genera una paleta completa de 12 tonos.

Cómo funciona

seed: #6750A4 (púrpura)

- genera 13 tonos (0, 5, 10, 20, 30, ..., 95, 100)
- asigna roles: primary, secondary, tertiary, surface, error

Rol	Uso	Ejemplo Flutter
primary	Botones principales, FAB, switches activos	colorScheme.primary
onPrimary	Texto/icono sobre primary	colorScheme.onPrimary
secondary	Botones secundarios, chips	colorScheme.secondary
surface	Fondos de pantalla, cards	colorScheme.surface
surfaceContainer	Cards, bottomsheets (nuevo en M3)	colorScheme.surfaceContainer
tertiary	Acentos alternativos	colorScheme.tertiary
error	Estados de error	colorScheme.error

Dynamic Color (Material You)

En Android 12+, el sistema puede generar el ColorScheme **desde el wallpaper del usuario**. Tu app se adapta automáticamente a los gustos del usuario.

```
En Android 12+ → ColorScheme.fromSeed(source: userWallpaperColor)
En otros       → ColorScheme.fromSeed(seedColor: brandColor)
```

Contraste

M3 garantiza contraste mínimo:

- Texto normal: 4.5:1
- Texto grande: 3:1
- Iconos: 3:1

2. Tipografía: type scale

M3 define una **escala tipográfica** con 11 estilos. No son estilos aislados, sino un **sistema** que escala armónicamente.

Style	Tamaño	Peso	Uso típico
displayLarge	57	Regular	Hero text
displayMedium	45	Regular	Pantallas de bienvenida
displaySmall	36	Regular	Encabezados grandes
headlineLarge	32	Regular	Títulos de sección
headlineMedium	28	Regular	Títulos de pantalla
headlineSmall	24	Regular	Subtítulos
titleLarge	22	Medium	App bar titles
titleMedium	16	Medium	Navigation, cards
titleSmall	14	Medium	Botones, tabs
bodyLarge	16	Regular	Cuerpo de texto
bodyMedium	14	Regular	Texto secundario
bodySmall	12	Regular	Captions, tags
labelLarge	14	Medium	Botones, chips
labelMedium	12	Medium	Texto en componentes
labelSmall	11	Medium	Textos pequeños

Regla de oro tipográfica M3

Usa los **nombres semánticos**, no tamaños fijos.



```
Theme.of(context).textTheme.titleLarge
```



```
TextStyle(fontSize: 22, fontWeight: FontWeight.w500)
```

3. Forma (Shape)

M3 define formas por **categoría de componente**, no globalmente.

Categoría	Esquinas típicas	Componentes
<code>small</code>	8px	Botones, chips, text fields
<code>medium</code>	12px	Cards, dialogs
<code>large</code>	16px	Bottom sheets, navigation drawers
<code>extraLarge</code>	20px	FAB grandes

En Flutter, las formas se definen mediante `ShapeBorder`:

```
Card(  
  shape: RoundedRectangleBorder(  
    borderRadius: BorderRadius.circular(12), // medium  
  ),  
)
```

4. Elevación

M3 cambió la forma de pensar la elevación. Ahora se usa **color en lugar de sombras** para indicar profundidad:

```
surfaceContainerLow    → nivel 1 (cerca del fondo)  
surfaceContainer       → nivel 2  
surfaceContainerHigh   → nivel 3 (cards levantadas)  
surfaceContainerHighest → nivel 4 (modales, dialogs)
```

Los contenedores de superficie (surface containers) permiten diferenciar capas sin añadir sombras artificiales.

5. Motion

M3 prioriza motion sutil y con propósito:

Principio	Descripción
Responsive	La UI reacciona al toque inmediatamente (ripple)
Expresivo	Las transiciones cuentan una historia (shared element transitions)
Familiar	Usa curvas estándar (emphasized, standard)

En Flutter, el ripple de M3 viene por defecto en todos los botones y componentes interactivos.

Resumen: M3 en una página

```
M3 Theme
├─ ColorScheme.fromSeed(seedColor: brandColor)
│   ├── light scheme
│   └─ dark scheme
├─ TextTheme (15 estilos, usa nombres semánticos)
└─ Shape scheme (small=8, medium=12, large=16)
```

Lo más importante que debes recordar:

- 1. **Elige un seed color** y deja que M3 genere el resto
- 2. *Usa colorScheme. siempre**, nunca colores hardcodeados
- 3. *Usa textTheme. siempre**, nunca tamaños fijos
- 4. **Prefiere surfaceContainer** sobre sombras para profundidad
- 5. **Soporta dark mode** desde el día 1 (M3 lo hace fácil)

Siguiente: 05 — Componentes M3 para Mobile

05 — Componentes M3 Clave para Mobile

El arsenal de UI para construir tu MVP

M3 clasifica sus componentes en 5 categorías. Aquí cubrimos los que más usarás en un MVP mobile.

1. Navegación

NavigationBar (antes BottomNavigationBar)

El estándar para navegación principal en mobile.

```
NavigationBar(  
  selectedIndex: 0,  
  onDestinationSelected: (index) {},  
  destinations: const [  
    NavigationDestination(icon: Icon(Icons.home), label: 'Inicio'),  
    NavigationDestination(icon: Icon(Icons.search), label: 'Buscar'),  
    NavigationDestination(icon: Icon(Icons.favorite), label: 'Favoritos'),  
    NavigationDestination(icon: Icon(Icons.person), label: 'Perfil'),  
  ],  
)
```

Reglas M3:

- Máximo 5 destinos
- Destino activo usa **primary** + tonal surface
- Muestra el label siempre (excepto en pantallas muy angostas)

NavigationDrawer

Para navegación secundaria o apps con muchas secciones.

```

Drawer(
  child: NavigationDrawer(
    selectedIndex: 0,
    onDestinationSelected: (index) {},
    children: [
      NavigationDrawerDestination(
        icon: Icon(Icons.inbox),
        label: Text('Inbox'),
      ),
    ],
  ),
)

```

TopAppBar

La barra superior. M3 tiene 3 variantes:

Variante	Uso
<code>AppBar</code> (centerTitle)	Pantallas principales
<code>AppBar</code> (small/leading)	Pantallas de detalle
<code>SliverAppBar</code>	Scroll con colapso

2. Contenido

Card

Contenedor para información agrupada.

```

Card(
  child: Column(
    children: [
      ListTile(
        leading: CircleAvatar(child: Icon(Icons.tennis)),
        title: Text('Cancha de Tenis'),
        subtitle: Text('Club Deportivo'),
      ),
      Row(
        mainAxisAlignment: MainAxisAlignment.end,
        children: [
          TextButton(onPressed: {}, child: Text('Reservar')),
        ],
      ),
    ],
  ),
)

```

Variantes: `Card` (elevated), `Card.outlined`, `Card.filled`.

ListTile

Ítem estándar para listas.

```
ListTile(  
  leading: Icon(Icons.location_on),  
  title: Text('Club Deportivo'),  
  subtitle: Text('A 2.3 km'),  
  trailing: Chip(label: Text('\$15/h')),  
)
```

3. Acción

Buttons

M3 tiene una jerarquía clara:

Tipo	Énfasis	Uso
FilledButton	Alto	Acción principal (Continuar, Pagar)
FilledTonalButton	Medio	Acción secundaria importante
OutlinedButton	Medio	Acción alternativa
TextButton	Bajo	Acción menos importante (Cancelar)

```
FilledButton(onPressed: () {}, child: Text('Reservar ahora'))  
FilledButton.icon(  
  icon: Icon(Icons.shopping_cart),  
  label: Text('Agregar al carrito'),  
)
```

FAB (FloatingActionButton)

Para la acción principal de la pantalla.

```
FloatingActionButton(  
  onPressed: () {},  
  child: Icon(Icons.add),  
)  
// Variante extendida (con texto):  
FloatingActionButton.extended(  
  onPressed: () {},  
  icon: Icon(Icons.add),  
  label: Text('Nueva reserva'),  
)
```

SegmentedButton

Alternativa moderna a RadioButton / ToggleButtons.

```
SegmentedButton<String>(  
  segments: [  
    ButtonSegment(value: 'day', label: Text('Día')),  
    ButtonSegment(value: 'week', label: Text('Semana')),  
    ButtonSegment(value: 'month', label: Text('Mes')),  
  ],  
  selected: selected,  
  onSelectionChanged: (v) {},  
)
```

4. Entrada de texto

TextField

```
TextField(  
  decoration: InputDecoration(  
    labelText: 'Buscar canchas',  
    prefixIcon: Icon(Icons.search),  
    filled: true,  
  ),  
)
```

SearchBar

Nuevo en M3. Más moderno que TextField para búsquedas.

```
SearchBar(  
  leading: Icon(Icons.search),  
  hintText: 'Buscar canchas...',  
  onSubmitted: (v) {},  
)
```

5. Feedback

SnackBar

Notificaciones temporales.

```
ScaffoldMessenger.of(context).showSnackBar(  
  SnackBar(content: Text('Reserva confirmada')),  
)
```

Badge

Indicadores de notificaciones o conteo.

```
Badge(  
  label: Text('3'),  
  child: IconButton(icon: Icon(Icons.notifications)),  
)
```

AlertDialog

Diálogos de confirmación.

```
AlertDialog(  
  title: Text('Confirmar reserva'),  
  content: Text('¿Reservar cancha por \$15/h?'),  
  actions: [  
    TextButton(onPressed: () {}, child: Text('Cancelar')),  
    FilledButton(onPressed: () {}, child: Text('Confirmar')),  
  ],  
)
```

Mapa de componentes para tu MVP

Usa esta tabla para decidir qué componente usar en cada parte de tu app:

Necesidad	Componente M3
Navegación principal	<code>NavigationBar</code> (inferior) o <code>NavigationDrawer</code> (lateral)
Barra superior	<code>TopAppBar</code>
Lista de ítems	<code>Card</code> + <code>ListTile</code>
Acción principal	<code>FilledButton</code> o <code>FAB</code>
Acción secundaria	<code>FilledTonalButton</code>
Selección única	<code>SegmentedButton</code> o <code>ChoiceChip</code>
Selección múltiple	<code>FilterChip</code>
Búsqueda	<code>SearchBar</code>
Formulario	<code>TextField</code> + <code>FilledButton</code>
Fecha	<code>DatePickerDialog</code>
Hora	<code>TimePickerDialog</code>
Confirmación	<code>AlertDialog</code> o <code>BottomSheet</code>
Feedback breve	<code>SnackBar</code>

Necesidad	Componente M3
Notificación	<code>Badge</code>
Progreso	<code>LinearProgressIndicator</code> o <code>CircularProgressIndicator</code>
Vacío / Error	<code>Center</code> + <code>Icon</code> + <code>Text</code> (diseño propio)

Regla de oro

Para cada pantalla del storyboard (del Design Sprint), identifica primero **qué necesidad de UI tiene** y luego ve a la tabla de arriba para elegir el componente M3.

Siguiente: 06 — Prototipado y Validación

06 — Prototipado + Validación

Día 4: Construir → Día 5: Testear con usuarios

Fase 5: Prototype

El objetivo es construir un **prototipo realista** del MVP en 1 día. No es código real, es una simulación convincente.

Opción A: Prototipado en Figma (recomendada para no-devs)

Figma + [Material 3 Design Kit](#).

```
Storyboard del Día 3
↓
Componentes M3 en Figma (arrastrar y soltar)
↓
Conexiones y flujos (prototyping mode)
↓
Prototipo navegable listo para test
```

Ventajas: Rápido (medio día), cualquier persona del equipo puede hacerlo. **Desventajas:** No muestra comportamiento real (scroll, carga, etc.).

Opción B: Prototipado en Flutter (recomendada para devs)

Usa el template de este módulo para construir el MVP funcional.

```
flutter create --template app mvp_prototipo
→ Copiar template theme/ de este módulo
→ Implementar las 3-4 pantallas del Golden Path
→ Datos mock (sin backend)
```

Ventajas: Comportamiento real, reutilizable como base del producto final. **Desventajas:** Más lento (1 día completo o más).

Opción C: Híbrida (recomendada)

```
Jueves AM: Figma → prototipo visual navegable (para testear el viernes)
Jueves PM: Flutter → empezar a codificar las pantallas (para producción)
```

Guía rápida: Prototipado en Figma con M3

1. **Instala el kit M3:** <https://www.figma.com/community/file/1035203688168086460>
2. **Define tu theme:** seed color → genera la paleta con [Material Theme Builder](#)
3. **Arma las pantallas:** arrastra componentes M3 (NavigationBar, Card, FilledButton, etc.)
4. **Conecta los flujos:** usa Figma Prototyping para enlazar pantallas
5. **Tip:** no diseñes todas las pantallas. Solo el Golden Path (3-4 pantallas) + estados clave (carga, error, vacío).

Fase 6: Validate

El día más importante del Sprint. Pones el prototipo frente a **5 usuarios reales** y observas.

¿Por qué 5 usuarios?

Con 5 usuarios encuentras ~85% de los problemas de usabilidad. Después del quinto, ves los mismos problemas una y otra vez.

Cómo reclutar usuarios

Método	Tiempo	Costo
Clients actuales	1-2 días	Gratis
Redes sociales	2-3 días	Gratis
UserInterviews.com	2-5 días	~\$50-100/usuario
Respondent.io	2-5 días	~\$100-200/usuario

Perfil de usuario ideal: alguien que **tiene el problema** que tu MVP resuelve.

Estructura de la entrevista (30 min)

5 min → Bienvenida y contexto ("cuéntame de ti")
5 min → Pregunta sobre el problema actual ("¿cómo reservas hoy?")
15 min → Test del prototipo ("ahora quiero que reserves una cancha")
→ Observa en silencio
→ No ayudes, no expliques
→ Pide que piense en voz alta
5 min → Cierre ("¿qué te pareció? ¿comprarías esto?")

Qué observar

Señal positiva	Señal de alerta
Navega sin ayuda	Se queda en blanco >5s

Señal positiva	Señal de alerta
Completa la tarea	Toca donde no hay botón
Dice "esto es justo lo que necesito"	Dice "¿y esto qué hace?"
Pregunta por precio	Pregunta "¿cómo vuelvo atrás?"
Sonríe/celebra al completar	Frunce el ceño, suspira

Matriz de resultados

Después de los 5 tests, clasifica los hallazgos:

	Impacto en el usuario	
	Bajo	Alto
Fácil de arreglar	Bajo / Fácil (mejora rápida)	Alto / Fácil (prioridad!)
Difícil	Bajo / Difícil (pospuesto)	Alto / Difícil (requiere rediseño)

Decisiones post-Sprint

Con los resultados, tienes 3 opciones:

Resultado	Acción
✔ Los usuarios completaron el flujo y mostraron interés	Pasar a desarrollo: el MVP está validado
✖ Los usuarios no completaron el flujo o no vieron valor	Pivotar: cambiar enfoque o solución
⚠ Resultados mixtos	Iterar: ajustar el prototipo y testear de nuevo (mini-Sprint de 2 días)

Checklist del Día 5

- ☐ 5 entrevistas completadas
- ☐ Grabaciones (audio/video) guardadas
- ☐ Matriz de hallazgos completada
- ☐ Decisión tomada: ✔ desarrollo / ↺ iterar / ✖ pivotar
- ☐ Próximos pasos documentados

Del prototipo validado al código Flutter

Una vez validado, el prototipo (Figma o Flutter) se convierte en el **contrato de diseño** para implementación.

En el siguiente capítulo verás cómo llevar el theme M3 a código Flutter.

Siguiente: 07 — M3 a Flutter: Implementación

07 — M3 a Flutter: Implementación

Traducir el sistema de diseño a código

Flutter implementa M3 de forma nativa. No necesitas paquetes adicionales.

Importante (Flutter 3.16+): desde la versión 3.16, **Material 3 es el valor por defecto** en `ThemeData`. Ya no es necesario (ni recomendado) activarlo con `useMaterial3: true`, pues ese flag está marcado para deprecación. Antes de la 3.16 sí era obligatorio; si migras un proyecto antiguo que aún usa Material 2, puedes desactivar M3 temporalmente con `useMaterial3: false` mientras completas la migración.

1. Configurar el tema M3

```
MaterialApp(  
  theme: ThemeData(  
    colorScheme: ColorScheme.fromSeed(  
      seedColor: Color(0xFF6750A4), // seed color  
    ),  
  ),  
  darkTheme: ThemeData(  
    colorScheme: ColorScheme.fromSeed(  
      seedColor: Color(0xFF6750A4),  
      brightness: Brightness.dark,  
    ),  
  ),  
  themeMode: ThemeMode.system, // respeta preferencia del sistema  
)
```

Con M3 activo (default desde Flutter 3.16):

- Los componentes usan los defaults M3 (NavigationBar, Cards, Buttons, etc.)
- Se activa la escala tipográfica M3
- Las formas cambian a las redondeadas de M3

2. ColorScheme

La clase central de color en M3.

Desde un seed color (recomendado)

```
ColorScheme.fromSeed(  
  seedColor: Color(0xFF6750A4),  
  brightness: Brightness.light,  
)
```

Esto genera **automáticamente** los 12 tonos. No necesitas definir nada más.

Esquema personalizado

```
ColorScheme(  
  primary: Color(0xFF6750A4),  
  onPrimary: Colors.white,  
  primaryContainer: Color(0xFFEADDFF),  
  onPrimaryContainer: Color(0xFF21005D),  
  secondary: Color(0xFF625B71),  
  onSecondary: Colors.white,  
  surface: Color(0xFFFFFBFE),  
  onSurface: Color(0xFF1C1B1F),  
  // ... más roles  
)
```

Usa `fromSeed` siempre que puedas. Personaliza solo cuando necesites un color exacto de marca.

Dynamic Color (Material You)

```
// En Android 12+  
ColorScheme.fromSeed(  
  seedColor: Color(0xFF6750A4),  
  dynamicSchemeVariant: DynamicSchemeVariant.tonalSpot,  
  brightness: Brightness.light,  
)
```

`DynamicSchemeVariant` controla el estilo de la paleta generada:

Variante	Efecto
<code>tonalSpot</code>	Default. Balanceado
<code>fidelity</code>	Se acerca más al color original
<code>monochrome</code>	Un solo tono
<code>neutral</code>	Colores neutros
<code>vibrant</code>	Más saturado
<code>expressive</code>	Colores llamativos

3. TextTheme

La escala tipográfica M3 en Flutter:

```
TextTheme(  
  displayLarge: TextStyle(fontSize: 57, fontWeight: FontWeight.w400),  
  headlineLarge: TextStyle(fontSize: 32, fontWeight: FontWeight.w400),  
  titleLarge: TextStyle(fontSize: 22, fontWeight: FontWeight.w500),  
  bodyLarge: TextStyle(fontSize: 16, fontWeight: FontWeight.w400),  
  labelLarge: TextStyle(fontSize: 14, fontWeight: FontWeight.w500),  
  // ... más estilos  
)
```

Uso en widgets:

```
Text('Título de la pantalla', style: Theme.of(context).textTheme.titleLarge)  
Text('Cuerpo del texto', style: Theme.of(context).textTheme.bodyMedium)
```

Font family personalizada

```
ThemeData(  
  textTheme: TextTheme(  
    titleLarge: TextStyle(fontFamily: 'Inter', fontSize: 22),  
    bodyMedium: TextStyle(fontFamily: 'Inter', fontSize: 14),  
  ),  
)
```

O global con `ThemeData(fontFamily: 'Inter')`.

4. ShapeScheme (formas)

En Flutter, las formas se definen por componente:

```
ThemeData(  
  cardTheme: CardTheme(  
    shape: RoundedRectangleBorder(  
      borderRadius: BorderRadius.circular(12), // medium  
    ),  
  ),  
  filledButtonTheme: FilledButtonThemeData(  
    shape: RoundedRectangleBorder(  
      borderRadius: BorderRadius.circular(8), // small  
    ),  
  ),  
)
```

5. Componentes M3

Una vez configurado el `ColorScheme` (con M3 como default), los widgets de Flutter ya usan M3:

Widget	Cambio con M3
<code>NavigationBar</code>	Reemplaza <code>BottomNavigationBar</code> . Más compacto, colores M3
<code>NavigationDrawer</code>	Reemplaza Drawer tradicional
<code>FilledButton</code>	Nuevo. Botón principal con fondo lleno
<code>FilledTonalButton</code>	Nuevo. Botón con fondo tonal
<code>SegmentedButton</code>	Nuevo. Reemplaza <code>ToggleButtons</code>
<code>SearchBar</code>	Nuevo. Barra de búsqueda M3
<code>Card</code>	Por defecto usa esquinas redondeadas M3
<code>Badge</code>	Nuevo. Indicador de notificaciones

6. ThemeExtensions (tokens personalizados)

Para valores que no cubre M3 (spacing, radii, custom shadows):

```
class AppSpacing extends ThemeExtension<AppSpacing> {
  final double xs;
  final double sm;
  final double md;
  final double lg;

  const AppSpacing({required this.xs, required this.sm, required this.md, required this.lg});

  @override
  AppSpacing copyWith({double? xs, double? sm, double? md, double? lg}) {
    return AppSpacing(xs: xs ?? this.xs, sm: sm ?? this.sm, md: md ?? this.md, lg: lg ?? this.lg);
  }

  @override
  AppSpacing lerp(AppSpacing? other, double t) {
    if (other == null) return this;
    return AppSpacing(
      xs: lerpDouble(xs, other.xs, t)!,
      sm: lerpDouble(sm, other.sm, t)!,
      md: lerpDouble(md, other.md, t)!,
      lg: lerpDouble(lg, other.lg, t)!,
    );
  }
}
```

Uso:

```
Theme.of(context).extension<AppSpacing>()!.md // → 16
```

7. Dark mode

Con `ColorScheme.fromSeed` y `Brightness.dark`, obtienes dark mode automático:

```
ThemeData(  
  colorScheme: ColorScheme.fromSeed(  
    seedColor: brandColor,  
    brightness: Brightness.dark,  
  ),  
)
```

No necesitas ajustar nada. Los componentes M3 se adaptan solos.

8. Arquitectura del theme (buena práctica)

Separa el theme en archivos independientes:

```
lib/theme/  
├─ app_theme.dart      → ThemeData light + dark  
├─ app_colors.dart     → ColorScheme (light + dark)  
├─ app_typography.dart → TextTheme  
└─ app_extensions.dart → ThemeExtensions (spacing, radii)
```

Esto es exactamente lo que incluye el template de este módulo.

Checklist de implementación M3

M3 es el default desde Flutter 3.16. No uses `useMaterial3: true` (deprecated). Solo usa `useMaterial3: false` si estás migrando un proyecto con Material 2.

- ☐ `ColorScheme.fromSeed` con seed color de marca
- ☐ Light + dark scheme definidos
- ☐ `textTheme` personalizado (fuente de marca si aplica)
- ☐ Componentes actualizados a M3 (NavigationBar, etc.)
- ☐ `ThemeExtensions` para tokens no cubiertos
- ☐ `ThemeMode.system` para respetar preferencia del usuario

Siguiente: 08 — Template de proyecto

08 — Template de proyecto

Starter Flutter con M3 + Clean Architecture para tu MVP

El directorio `template/` contiene un proyecto Flutter listo para usar como base de tu MVP. Incluye:

- M3 theming completo (light + dark + dynamic color)
- Clean Architecture (data/domain/presentation)
- Componentes M3 reutilizables
- Estructura preparada para escalar

Estructura

```
template/
├── lib/
│   ├── main.dart           # Entry point
│   ├── app.dart            # MaterialApp con M3
│   └── theme/
│       ├── app_theme.dart  # ThemeData light + dark
│       ├── app_colors.dart # ColorScheme.fromSeed
│       ├── app_typography.dart # TextTheme M3
│       └── app_extensions.dart # Spacing, Radii tokens
│   └── features/
│       └── mvp/             # Feature de ejemplo
│           ├── data/
│           │   ├── datasources/ # Fuentes de datos (API, local)
│           │   └── repositories/ # Implementación de repositorios
│           ├── domain/
│           │   ├── entities/    # Modelos de dominio
│           │   ├── repositories/ # Contratos (abstract)
│           │   └── usecases/    # Casos de uso
│           └── presentation/
│               ├── pages/       # Pantallas
│               └── widgets/     # Widgets reutilizables
├── core/
│   └── components/          # Componentes M3 compartidos
├── test/
│   └── ...                  # Tests unitarios + de widgets
└── pubspec.yaml
```

Cómo usar el template

Opción 1: Copiar sobre un proyecto existente

```
# Después de flutter create
cp -r template/lib/theme/ tu-proyecto/lib/theme/
cp -r template/lib/core/ tu-proyecto/lib/core/
# Adapta main.dart y app.dart
```

Opción 2: Usar como base

```
cp -r template/ mi-mvp/
cd mi-mvp
# Edita pubspec.yaml con tu nombre, dependencias, etc.
flutter pub get
```

Opción 3: Referencia de estudio

Lee cada archivo del template como ejemplo de implementación M3 + Clean Architecture.

Personalización rápida

1. Cambia el seed color en `app_colors.dart`:

```
static ColorScheme light() => ColorScheme.fromSeed(
  seedColor: Color(0xFFTU_COLOR), // ← tu color de marca
);
```

2. Agrega tu font en `app_typography.dart` y `pubspec.yaml`:

```
// app_typography.dart
TextTheme(
  titleLarge: GoogleFonts.interTextTheme().titleLarge,
  bodyMedium: GoogleFonts.interTextTheme().bodyMedium,
)
```

3. Agrega tus pantallas en `features/mvp/presentation/pages/`:

```
class HomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Mi MVP')),
      body: Center(child: Text('¡A construir!')),
    );
  }
}
```

Dependencias incluidas

En `pubspec.yaml`:

```
dependencies:
  flutter:
    sdk: flutter
  google_fonts: ^8.2.1 # tipografía M3 fácil

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0
```

Integración con otros módulos

Necesidad	Módulo relacionado
State management (BLoC/Cubit)	16-BLOC-CUBIT
Backend con Supabase	03-SUPABASE
Almacenamiento local	04-ALMACENAMIENTO-LOCAL
Testing	05-TESTING
Widgets avanzados	15-WIDGETS-FLUTTER

Siguiente: 09 — Caso completo MVP

09 — Caso completo: App de Reservas

De Design Sprint a Flutter con M3 en 5 días

Este caso integrador recorre el proceso completo con un ejemplo real: una app MVP para **reservar canchas de tenis**.

Día 1: Understand + Define

Goal a largo plazo

"Los usuarios pueden reservar una cancha de tenis en menos de 30 segundos desde su teléfono, sin llamar por teléfono."

Métrica principal: Tiempo promedio de reserva < 30 segundos.

Mapa del problema

```
[Usuario quiere jugar tenis]
  → Google Maps "canchas de tenis cerca"
  → Encuentra teléfono
  → Llama → Ocupado → Llama de nuevo
  → "¿Tienen cancha libre a las 5pm?"
  → "Sí" → "OK, reserve"
  → Llega → No está reservada → Enfado
  ← Pierde una hora de juego ←
```

HMWs seleccionados

- "¿Cómo podríamos mostrar disponibilidad en tiempo real?"
- "¿Cómo podríamos permitir reservar sin hablar con nadie?"
- "¿Cómo podríamos confirmar la reserva instantáneamente?"

Golden Path

1. Home → buscar canchas (filtro por fecha/hora)
2. Lista de canchas disponibles → seleccionar
3. Confirmar datos → pagar
4. Confirmación → ticket digital

Alcance del MVP

Incluye	No incluye
Búsqueda por fecha/hora	Registro de usuario obligatorio
Lista de canchas con precio	Historial de reservas
Reserva y pago	Favoritos
Confirmación	Chat con el club
Modo invitado (sin registro)	Reseñas

Sprint Goal

"Validar que un usuario puede encontrar y reservar una cancha de tenis en menos de 30 segundos sin ayuda."

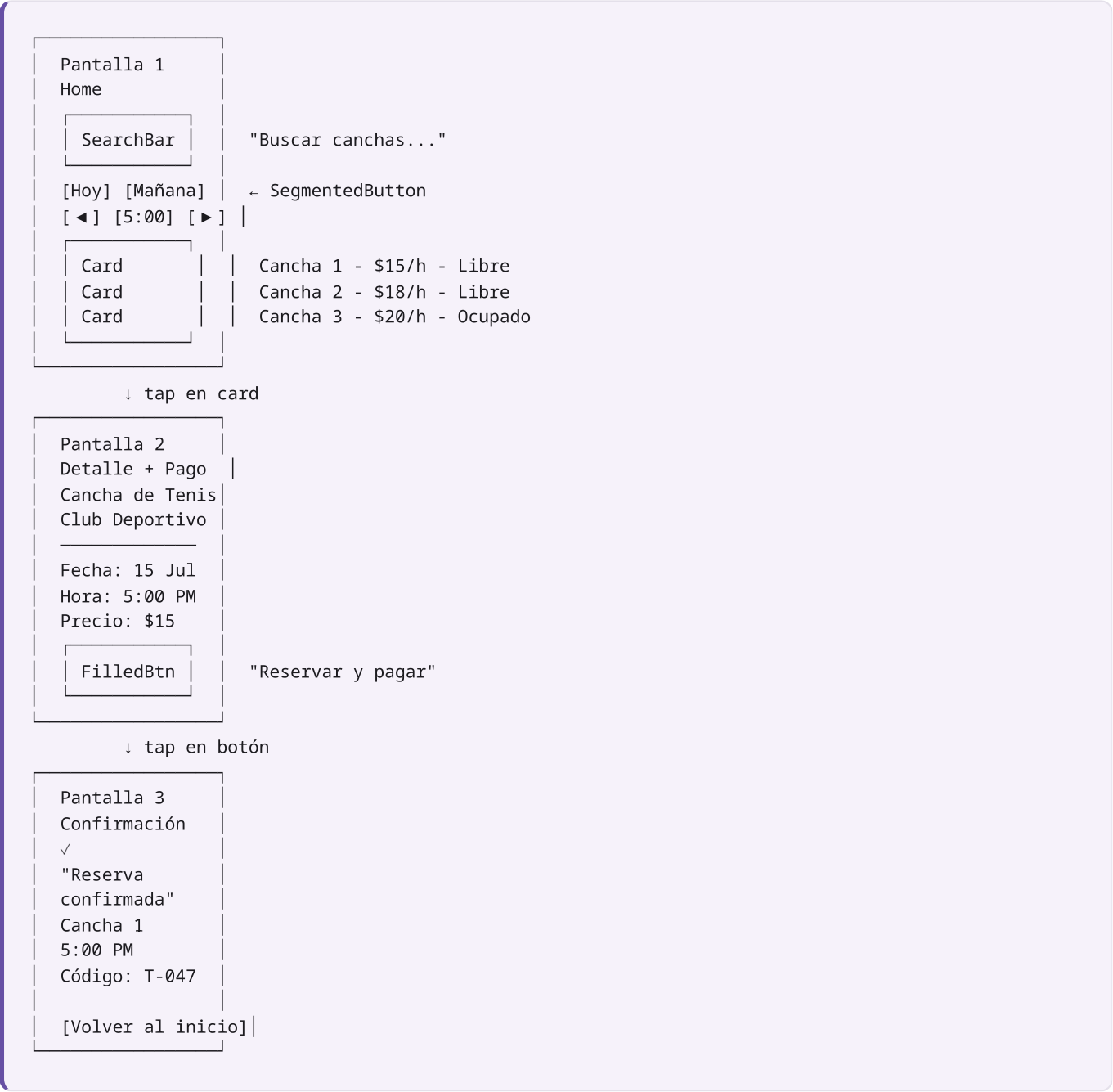
Día 2: Sketch

Solution Sketch (por el diseñador)

Panel 1: Usuario frustrado llamando por teléfono sin respuesta. Panel 2: App con campo de búsqueda, calendario y botón "Reservar". Panel 3: Usuario feliz con confirmación en pantalla.

Día 3: Decide

Storyboard final



Componentes M3 identificados

Pantalla	Componentes M3
Home	SearchBar , SegmentedButton , Card + ListTile
Detalle	Card , FilledButton , ListTile
Confirmación	Card , Icon (check), TextButton
Estados	CircularProgressIndicator , SnackBar , AlertDialog

Día 4: Prototype (Figma + Flutter)

Theme M3 definido

```
Seed color: #1B5E20 (verde oscuro, evoca cancha de tenis)
→ Light + Dark scheme generados con ColorScheme.fromSeed
→ Tipografía: Google Fonts Inter
→ Shape: 8px small, 12px medium, 16px large
```

Pantallas en Figma

Se crean las 3 pantallas del storyboard usando componentes M3 del kit de Figma. Flujos conectados: Home → Detalle → Confirmación.

Base en Flutter (template)

Se copia el template y se configura el theme con el seed color verde.

Día 5: Validate

Usuarios reclutados

5 personas que juegan tenis al menos 1 vez por semana.

Resultados

Usuario	Completó la tarea?	Tiempo	Problemas
1	✔ Sí	25s	Ninguno
2	✔ Sí	20s	Dudó en el SearchBar (no obvio)
3	✔ Sí	35s	Quería filtrar por precio
4	✘ No	60s	No encontró el botón de reserva
5	✔ Sí	28s	"Muy fácil, me encantó"

Hallazgos

Hallazgo	Prioridad	Acción
SearchBar no es obvio como buscador	Alta	Agregar hint text + icono de lupa más claro

Hallazgo	Prioridad	Acción
Falta filtro por precio	Media	Agregar como post-MVP
Botón de reserva muy abajo	Alta	Mover FilledButton arriba, hacerlo sticky
El flujo general funciona	-	MVP validado, pasar a desarrollo

Decisión

✅ **Pasar a desarrollo.** Los 3 de 5 usuarios completaron la tarea en <30s. Los problemas identificados son fáciles de corregir antes del lanzamiento.

De vuelta a Flutter: implementación del MVP

Con el prototipo validado y los hallazgos corregidos, se implementa usando:

1. **Template M3** → theme con seed color + dark mode
2. **Clean Architecture** → features/mvp con data/domain/presentation
3. **Datos mock** → sin backend todavía (los repositorios devuelven datos estáticos)
4. **Golden Path** → las 3 pantallas conectadas con navegación

```
// app.dart
MaterialApp(
  theme: AppTheme.light,
  darkTheme: AppTheme.dark,
  themeMode: ThemeMode.system,
  home: HomePage(),
)
```

```
// features/mvp/presentation/pages/home_page.dart
class HomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Reservar cancha')),
      body: Column(
        children: [
          SearchBar(hintText: 'Buscar canchas...'),
          SegmentedButton(...),
          Expanded(
            child: ListView(
              children: canchas.map((c) => Card(
                child: ListTile(
                  title: Text(c.nombre),
                  subtitle: Text('\${c.precio}/h'),
                  trailing: c.disponible
                    ? FilledButton(onPressed: () {}, child: Text('Reservar'))
                    : Chip(label: Text('Ocupado')),
                ),
            ).toList(),
          ),
        ],
      ),
      bottomNavigationBar: NavigationBar(destinations: [...]),
    );
  }
}
```

Lecciones del caso real

1. **El Design Sprint ahorró semanas:** detectamos el problema del botón de reserva antes de escribir código.
2. **M3 aceleró el diseño:** no diseñamos desde cero, usamos componentes ya definidos.
3. **El template M3 + Clean Arch:** nos permitió pasar de prototipo a código en horas.
4. **La validación con 5 usuarios fue suficiente:** encontramos los problemas críticos.

Siguiente: BIBLIOGRAFIA

Bibliografía y recursos oficiales

Design Sprint

Recurso	Tipo	Enlace
GV Design Sprint (Jake Knapp) — guía oficial del proceso	Guía + métodos	https://www.gv.com/sprint/
The Design Sprint (libro)	Libro	https://www.gv.com/sprint/
Google Design Sprint Kit (histórico, sin soporte activo)	Guía + métodos archivada	https://designsprintkit.withgoogle.com/methodology
The Design Sprint (videos)	Videos	https://www.youtube.com/playlist?list=PLI5pVb4v-8nPgI7wHmR0y3OcE1G5r5z0y

Material Design 3

Recurso	Tipo	Enlace
Material Design 3 (oficial)	Sistema completo	https://m3.material.io
Foundations (color, typography, shape)	Guía	https://m3.material.io/foundations
Material Theme Builder	Herramienta	https://m3.material.io/theme-builder
Material 3 Figma Kit	Kit Figma	https://www.figma.com/community/file/1035203688168086460
Componentes M3 (catálogo A-Z)	Glosario	https://m3.material.io/components

Flutter + M3

Recurso	Tipo	Enlace
Material Design para Flutter (oficial)	Guía	https://docs.flutter.dev/ui/design/material
MDC-101: Material Components Basics	Codelab	https://codelabs.developers.google.com/codelabs/mdc-101-flutter

Recurso	Tipo	Enlace
MDC-102: Material Structure and Layout	Codelab	https://codelabs.developers.google.com/codelabs/mdc-102-flutter
MDC-103: Material Theming (Color, Shape, Type)	Codelab	https://codelabs.developers.google.com/codelabs/mdc-103-flutter
Flutter App Architecture Guide	Guía	https://docs.flutter.dev/app-architecture
Flutter Widget Catalog	Referencia	https://docs.flutter.dev/reference/widgets

Diseño mobile (iOS/Android)

Recurso	Tipo	Enlace
Apple Human Interface Guidelines	Guía	https://developer.apple.com/design/human-interface-guidelines
Android Design for Mobile	Guía	https://developer.android.com/design/ui/mobile

Libros recomendados

Título	Autor	Por qué
<i>Sprint</i>	Jake Knapp	El libro original del Design Sprint
<i>The Design of Everyday Things</i>	Don Norman	Fundamentos de diseño centrado en el usuario
<i>Don't Make Me Think</i>	Steve Krug	Usabilidad web y mobile, práctico y corto
<i>Material Design 3: Documentation</i>	Google	Leer online, no comprar (es gratis)